

Comparison of different algorithms for time analysis for CPM schedule networks

Levente Malyusz^a, Miklos Hajdu^{a,b,*}, Zoltan Vattai^a

^a Budapest University of Technology and Economics, Budapest, Hungary

^b Chaoyang University of Technology, Taiwan

ARTICLE INFO

Keywords:

Critical path method
Precedence diagramming method
Time analysis

ABSTRACT

Traditional Critical Path Method (CPM) and its different generalizations have become the prevailing technique in managing complex projects over recent decades. Among them, Precedence Diagramming Method (PDM), with its four precedence relationships, has become the most popular due to its increased flexibility in modeling complex project logic. Despite the four well-known precedence relationships, CPM/PDM still suffers from modeling shortcomings.

Developments such as conditional logic, maximal relationships, point-to-point relationships, continuous relationships, and non-linear activities attempt to remedy these problems. However, these improvements require more complex and sometimes iterative time analysis.

Monte Carlo (MC) simulation, which calculates the effects of risks on the project duration, also requires numerous repetitions of the deterministic time analysis. It is especially true if the network contains the developments mentioned above. In these cases, the time analysis speed is of great importance.

The goal of this study is a) to implement different algorithms for time analysis that perform better than the traditional time analysis, b) to compare their speed on large-scale artificially created projects, c) to make suggestions regarding which algorithms are suitable in different cases d) to develop a database containing artificially created networks to serve as a basis for benchmarking for researchers and developers working in the field of project planning.

Comparisons were based on networks with at least 1000 activities. Results show that, depending on the sample network's structure, different algorithms perform well. Therefore, different time analysis algorithms are necessary to implement into the scheduling tools, and they can decide which algorithms perform better.

1. Introduction and justification for the research

Algorithms for time analysis of PDM schedule networks are our investigation's objects. The time analysis that defines the minimal and yet feasible project duration and the early/late starts/finishes of the activities is easy and fast when only the four traditional precedence relationships (SS, SF, FS, FF) are used. This algorithm is part of all construction management curricula and will be referred to as Algorithm #1. However, there are some improvements to the technique (e.g., maximal precedence relationships, bi-directional relationships), and they require more complex time analysis. Some scheduling problems, for instance, finding the project duration's distribution by performing Monte Carlo analysis, require the repetition of the simple or the more complex time analysis numerous times, enlarging the running time by

order of magnitudes.

This study's general aim is to make proposals about which algorithms perform better on construction type networks. In the following, maximal relationships, bi-directional relationships, and Monte Carlo simulation are discussed briefly to justify the need for faster algorithms for time analysis.

1.1. Maximal relationships

Contrary to the traditional (minimal) precedence relationships, maximal precedence relationships define the maximum allowable time between the activities' connected points. For instance, activity B can start three (or more) days after finishing activity A. However, due to some reasons, this difference cannot be more than five days (Fig. 1a).

* Corresponding author at: Budapest University of Technology and Economics, Budapest, Hungary.

E-mail address: miklos.hajdu63@gmail.com (M. Hajdu).

<https://doi.org/10.1016/j.autcon.2021.103697>

Received 19 June 2020; Received in revised form 12 March 2021; Accepted 2 April 2021

Available online 23 April 2021

0926-5805/© 2021 The Authors.

Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license

(<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

The maxFS5 relationship between A and B ensures that B will commence within five days after the finish of A by pulling activity A closer to B if other activities and relationships would force B to start more than five days after the finish of activity A. Unfortunately, the well-known time analysis (Algorithm #1) can handle only minimal relationships. Therefore, before commencing the time analysis, these maximal relationships have to be converted into equivalent minimal relationships (Fig. 1). The reader can see that the only difference between a maximal and a minimal relationship is the inequality sign's direction (Fig. 1b). Multiplying both sides of the inequality for the maximal relationship by negative one (−1) reverses the inequality sign's direction and creates a minimal relationship. The transformed minimal relationship connects the same two points of the activities; however, it goes in a reverse direction and has a negative lag (Fig. 1d). Unfortunately, this transformation can create loops (Fig. 1d). Algorithm #1 is not suitable in these cases since it assumes loopless networks. Instead, different iterative, therefore, slower algorithms are applied. Three algorithms that handle maximal relationships are discussed and referred to as Algorithm #2- #4.

1.2. Bi-directional relationships

Conditional logic, such as bi-directional relationships, dramatically enhances the technique's modeling capability. The following simple example justifies this statement.

Two houses are under construction close to each other. Three different tasks follow each other consecutively on each house: A- structural works, B- mechanical and electrical works, and C- finishing works. There are three specialized teams: Team A for executing A1 and A2, Team B for B1 and B2, and Team C for C1 and C2. Table 1 shows the activity durations.

Only one team can work on one site at a time. Furthermore, a team can work only on one project at a time. This latter condition implies that activities requiring the same resources (same teams) cannot overlap. Adding FS0 relationships between A and B and B and C on both sites satisfies the first requirement. Satisfying the requirement of no overlapping is possible by adding FS0 relationships between similar activities. (See the vertically drawn FS0 relationships in Fig. 2a–h.)

However, one can define these FS0 relationships differently, e.g., A1 follows A2, or A2 follows A1. The same is valid for teams B and C. These different team sequences lead to eight different network logics.

Fig. 2 a–h shows the eight possible networks together with the early dates, activity duration, project duration, and critical path (critical relationships are in bold). Planners looking for the shortest project duration have to create and analyze all these schedule networks. With the recently developed bi-directional relationships, all these possibilities can be modeled in one network (Fig. 3).

A bi-directional FS0 relationship between, for example, A1 and A2 defines a conditional logic: either A2 follows A1, or A1 follows A2. The algorithm, handling bi-directional relationships (Fig. 3), systematically produces and then analyzes all the possible arrangements and selects the one with the shortest project duration.

Table 1

Duration of activities on the streets.

	Activity A	Activity B	Activity C
House #1	A1, 5 days	B1, 2 days	C1, 6 days
House #2	A2, 4 days	B2, 7 days	C2, 5 days

The problem described above seems the same as the repetitive scheduling problem, which has been thoroughly studied over the last decades. Network techniques using bi-directional relationships can model the constraints of repetitive scheduling problems; however, the goals are different. According to Ioannou and Yang [21], the goal of repetitive scheduling problems is "... to schedule activities deliberately so that each crew may work continuously from unit to unit without interruptions." The priority in repetitive scheduling problems is to minimize teams' idle times, while in network models, one seeks to minimize the project duration. Bi-directional relationships offer much more than modeling repetitive scheduling in schedule networks. Some of the key features of using bi-directional relationships are:

- They can model the constraints of repetitive scheduling problems and their different extensions. For example, if teams' continuous work is unnecessary, minimal bi-directional FS0 relationships are sufficient to describe the logic between the similar activities. If continuous work is a requirement, then both minimal and maximal bi-directional FS0 relationships need to be added to the logic. If continuous work on sites is necessary, then min and max bi-directional FS0 relationships need to be added to the project logic. If continuous work on a site is unnecessary, then a simple bi-directional FS0 relationship with greater than zero lag is sufficient.
- The time analysis used for bi-directional relationships seeks the shortest possible project duration. Repetitive scheduling algorithms usually seek to ensure work continuity for crews. The two objective functions often contradict, leading to different solutions for the same constraints.
- Bi-directional precedence relationships are also applicable for planning non-repetitive projects. For example, thousands of activities are to be executed, while only a few require the same resources (teams).
- Bi-directional relationships are not necessarily FS0 relationships. For example, two machines are working on the same linear site, and the safety distance has to be 50 m between them at all times. It is irrelevant which goes first; however, maintaining the safety distance is necessary.

1.3. The Monte Carlo simulation

The Monte Carlo methods are a broad class of algorithms to solve stochastic problems by creating ample deterministic instances using random sampling. In schedule networks, one can calculate the temporal effects of the risks affecting the activity durations on the project duration. During the Monte Carlo simulation, the algorithm generates

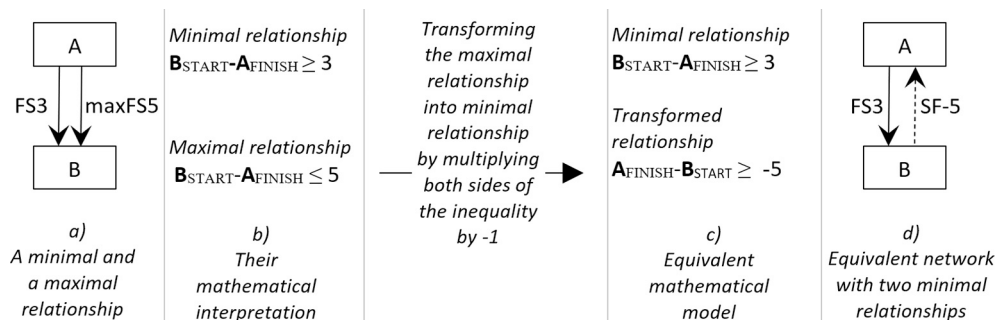


Fig. 1. Maximal relationships and their equivalent minimal relationships.

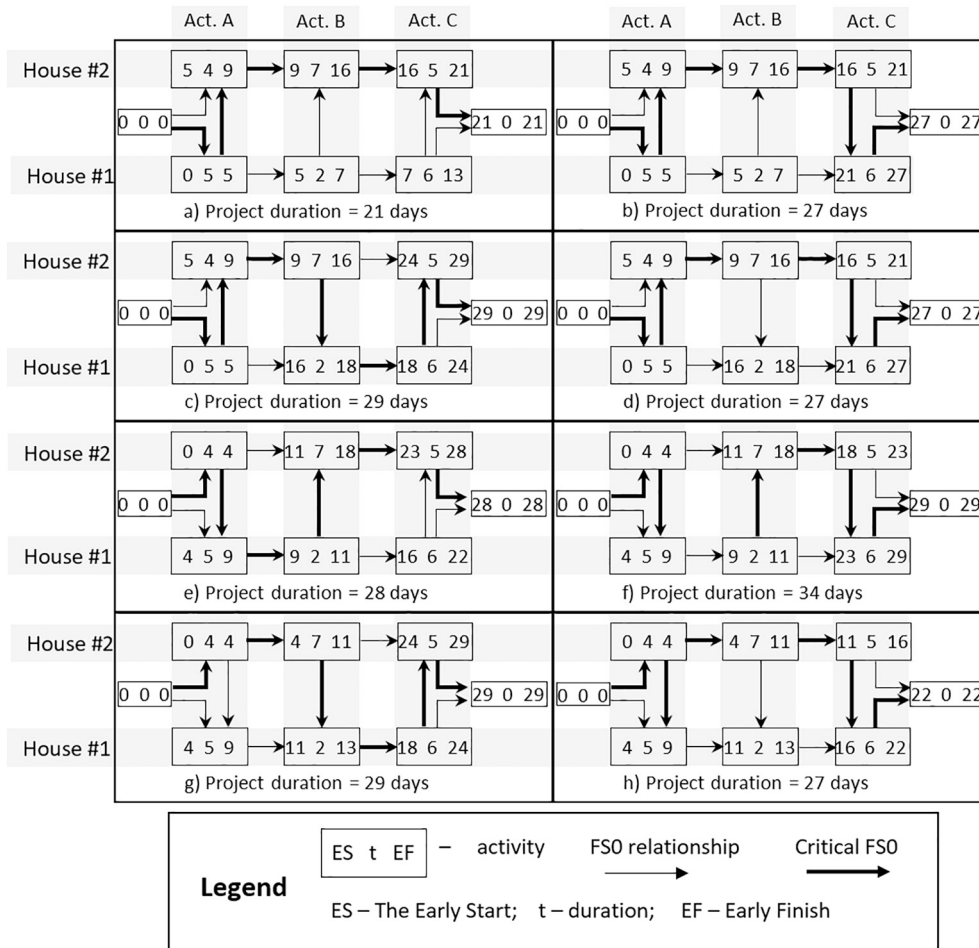


Fig. 2. Possible project networks for the sample project.

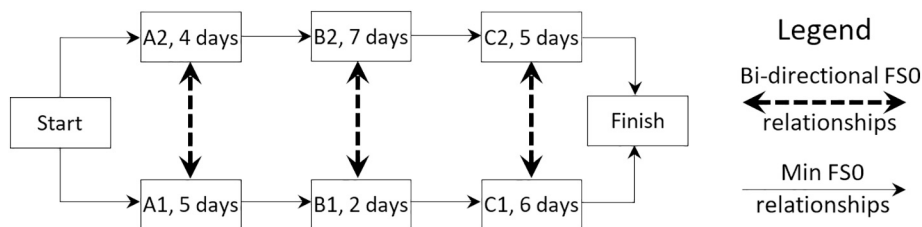


Fig. 3. Using bi-directional relationships to model eight different project logics in one network.

random activity durations (based on the previously defined activity distributions) for all activities. After creating a new instance, the software performs time analysis and repeats this two-step process to create the project duration's empirical distribution. Performing Monte Carlo simulation on a network with maximal relationships and conditional logic perfectly justifies the need for studying the speed of the simple time analysis.

To conclude, it can be stated that there are methods and improvements of the PDM technique requiring numerous repetitions of the simple or the complex time analysis, which perfectly justifies the need for this research.

2. Literature review

The complete time analysis (forward and backward pass) that defines the project duration and the early/late start/finish dates for all activities for CPM network schedules was developed in 1957 and introduced to

the scientific community in 1959 [22,23]. The more flexible PDM results from Roy [29], Fondahl [7], and IBM [20], among others. The original PDM technique used the activity-on-node (AoN) representation for displaying the schedule network. It allowed four precedence relationships for defining the minimum allowable distances between the end-points of the related activities. Roy's technique was called the 'method of potentials' or Metra Potential Method (MPM). According to Fondahl's reminiscences on the history of PDM [8], the term 'Precedence Diagramming Method' was first used by the IBM team.

Point-to-Point (PtP) relationships that can connect any two points of the related activities are results of Francis & Miresco [9], Kim [25], Hajdu [13].

Many algorithms transform traditional precedence relationships into SS relationships [24] [6] before time analysis. Transforming PtP relationships into equivalent SS relationships also seems to be a practical approach [13]. Maximal SS relationship in its minimal equivalent form (Fig. 1d) results from Roy [29].

Maximal relationships in their present form (Fig. 1a) and the iterative algorithm referred to Algorithm #2 result from Hajdu [12]. The maximal version of PtP relationships and the corresponding time analysis also result from Hajdu [14].

Algorithms #2-#4 are iterative algorithms used when maximal relationships are in the network. Algorithm #2, developed by Hajdu [12], can be seen as a modified version of the shortest path algorithm of Bellman [4] and Ford [10] for schedule networks. The modified FIFO algorithm (Algorithm #3), which is explained later, is based on the idea presented by Moore in 1959 [30] and Bellman [4] for finding the shortest path on non-directed networks. The DEQUEUE implementation used in Algorithm #4 uses the idea of D'Esopo & Pape [30]. Algorithm #3 and #4 have the same theoretical complexity as Algorithm #2 but are thought to be faster in linear-construction-like networks.

Algorithms #5/a-c are designed explicitly for Monte Carlo analysis based on algorithms #2-#4. In the first instance, they use Algorithm #2-#4, and they save the relationships' calculation order. During the second phase, which involves all the remaining instances, the calculation uses the calculation order defined during the first instance, making the time for calculating a new instance much faster.

Several articles have compared different shortest path algorithms for different applications [2,11,32,33]; however, none of them investigated project schedules. This paper aims to study the characteristics of Algorithms #2-#4 and their modification developed for Monte Carlo simulation (Algorithm #5 a-c) on construction-like networks.

Vanhoucke and Coelho [34] developed bi-directional relationships for modeling scarce resources. The time analysis handling bi-directional relationships results from Hajdu [17]. Monte Carlo analysis for solving the PERT problem on AoA network results from van Slyke [35]. Repetitive scheduling and its difference from CPM is discussed in many papers. [21] [27]. Unlike its original objective, repetitive scheduling today comprises many different optimization problems [19] [18]. However, only bi-directional relationships make the problem modelable in schedule networks.

Mathematicians often refer to the time analysis of a schedule network as a modification of the Dijkstra [5] algorithm. This algorithm was developed to find the shortest path between two given nodes on undirected graphs. However, CPM time analysis had already been devised at that time, although it can be seen as a version of Dijkstra's algorithm using a specific graph structure.

Publicly available databases for testing project management related algorithms are hard to find except the Project Scheduling Problem Library (PSPLIB) [26], which is used for testing algorithms dealing with resource limitations. A library with artificial construction like networks called PNLIB was created to suit the requirement of testing the time analysis algorithms on large scale construction networks.

3. Research aims and limitations

AoN schedule networks with linear, non-stretchable, and non-splittable activities with single and bi-directional minimal/maximal point-to-point relationships are the subject of this study. This research aims to decide which algorithms perform better for time analysis in large scale linear construction projects. The following research objectives have been set to satisfy the goal:

- 1) Develop the computer implementation for the five algorithms listed above
- 2) Develop a tool for generating large scale construction-like artificial networks
- 3) Develop a benchmark database based on the results of research of objective #2
- 4) Comparing the speed of the algorithms using the example projects (see objective #3)
- 5) Evaluating the results and making recommendations.

For simplicity, some limitations were assumed regarding the schedule network. These are the following:

- The network is deterministic
- Activities are linear, non-splittable, and non-stretchable
- WBS is not defined (there are no summary activities)
- Minimal and maximal single and bi-directional PtP relationships are allowed. (They are all converted into minSS relationships before starting the time analysis)
- Continuous relationships [15,16] are not allowed
- General OR relationships [17] are not allowed
- Calendars and imposed dates (constraints) are not allowed
- Hammock activities are not allowed
- Resources are assumed to be unlimited

4. Research methodology

For satisfying research objective #1, the engineering and mathematical models are presented together with the pseudo-codes governing the five different algorithms. These topics are presented in the following sections: 'The engineering model,' 'The mathematical model,' and 'The algorithms.' The algorithms were implemented in a Visual Studio environment and were written in C++.

For satisfying research objective #2, a tool called Project Network generator (PNGEN) has been developed. This network generator can generate a network of any size and topology; therefore, it can also generate construction networks. Twenty highway and railway schedule networks have been analyzed to capture some of the essential characteristics of construction networks. Three indices were defined as the result of this work.

To the best of the authors' knowledge, databases for testing time analysis algorithms are not available. A database called Project Network Library (PNLIB) was created containing artificial construction-like networks to fulfill research objective #3.

For satisfying research objective #4, each network has been run 1000 times, and the averages of the running duration were expressed in nanoseconds. All the test runs were performed on the same computer to make the results comparable.

For satisfying research objective #5, the results were compared, and suggestions were made on the algorithm's applicability for different cases.

The rest of the paper was structured to follow the flow of research objectives: Sections 'The engineering' model, 'The mathematical model' and 'The algorithms' satisfy research objective #1. Sections 'The topology of construction networks' and 'Introducing the Project Network Generator (PNGEN)' satisfy research objective #2. Section 'The Project Network Library (PNLIB) introduces the artificial and real-life networks used for testing to fulfill research objective #3. The following section, 'Comparing the algorithms', contains and compares the test results, thus fulfilling research objective #4. Section 'Discussion' covers research objective #5. The paper ends with a section called 'Conclusions and further research.'

5. The engineering model

Answers for the three questions (project duration, early dates for the activities, and late dates for the activities) of time analysis are sought for schedule networks. There are two special activities: one represents the start of the project, and the other represents the project's finish. Activities are assumed to be linear, continuous, non-stretchable, which means that any two of the start dates, the duration, and the finish date describe the activity's temporal realization.

An integer, non-negative z duration expresses the minimal necessary temporal distance between the logically connected points. (If these points are the finish points of the predecessor and the start of the successor, then the relationship is called the FS z relationship.) Moreover,

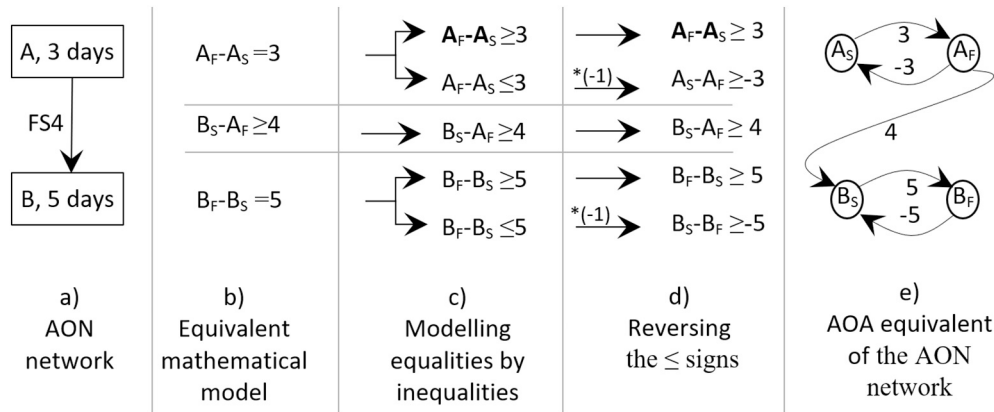


Fig. 4. An AoN activity network and its mathematical equivalent AoA graph.

the maximal allowable time difference can also be defined between the activities' connected points. The number of precedence relationships between any two activities is not restricted. The graphs have to be acyclic before transforming maximal relationships into minimal relationships. For simplicity, calendars are not allowed in the project. This structure can be displayed as an AoN graph where nodes stand for activities, and directed edges represent the logic. Loops and calendars are not allowed in this graph. This enhanced engineering model is referred to as CPM, PDM, or MPM network, although it is more general regarding its modeling capabilities than its predecessors.

The AoN is an engineering term, which, mathematically, can be interpreted as a graph where zero value loops represent the activities (Fig. 4). Fig. 4a shows a simple PDM network in the form of an AoN network, while Fig. 4e shows its equivalent using the AoA representation. One can follow the steps of this transformation by studying Fig. 4b–d. Fig. 4b shows the mathematical constraints for the activities and the precedence relationship. Fig. 4c presents the equations representing the activities transformed into inequalities. Fig. 4d represents the same logic with the reverse direction of $\leq$ signs, and Fig. 4e is the AoA representation of Fig. 4d, consequently the AoA equivalent form of the AoN network of Fig. 4a.

Time analysis in deterministic project networks with only minimal precedence relationships uses non-iterative algorithms such as Algorithm #1, although the mathematically more complex, therefore slower, iterative algorithms can also be used.

6. The mathematical model

For the applied algorithms, PDM networks will be transformed to directed graphs where nodes represent the activities' starts. Fig. 5 shows the steps of this transformation. First, relationships are converted into SS relationships (Fig. 5b). Fig. 5c shows the AoA representation of Fig. 5b. Fig. 5d presents the simplified network without the nodes representing the activity finishes.

The following mathematical model can be established based on Fig. 5d.

Given $[N, A, d]$, where $N = \{1, 2, \dots, i, \dots, n\}$ is the set of nodes of the graph, with two distinguished nodes; one represents the start (s), the other represents the finish (t) of the project, and A is the set of directed arcs of that graph. Let d_i stand for the length (duration) of the arcs representing the minSS equivalents of the relationships between i and j in the engineering model. Every node is the element of at least one path starting from s and finishing at t . Cycles in the graph are allowable.

Let us find a directed path $P(s \rightarrow t)$ from s to t in this graph so that the length of the path be maximum.

$$L(P) = \sum_{i=1}^k d_{i-1,i} \quad (1)$$

This problem is called the primal problem. A dual problem can be defined for this primal problem.

Let a μ_i integer value be assigned to each node (these represent the start dates in the engineering model). In this nomenclature, μ_s represent the start date of the start activity, while μ_t stands for the finish activity's start date. The goal is to find a μ_i policy for all $i \in N$ that minimizes μ_t while satisfying (2 and 3).

$$d_{ij} \leq \mu_j - \mu_i \quad \forall (i,j) \in A \quad (2)$$

$$\mu_s = 0 \quad (3)$$

$$\min \mu_t \quad (4)$$

The solution is optimal when the objective functions have the same value. Many possible feasible time policies satisfy the dual problem. The policy with the smallest values is called the early time policy. It gives the early start of the activities of the engineering model. The time policy with the largest values is called the late time policy, and the μ_i values, in this case, stand for the late start times in the engineering model. Early dates are defined during the so-called forward calculation; late dates result from the so-called backward pass.

Five algorithms have been implemented during this research:

- Algorithm #1: this can be applied if only minimal relationships are allowed. The time analysis developed by Kelley and Walker is implemented.
- Algorithm #2: this algorithm can handle acyclic graphs, the results of transforming maximal relationships into minimal ones.
- Algorithm #3: modified version of Algorithm #2 (with FIFO implementation)
- Algorithm #4: modified version of Algorithm #2 (with DEQUEUE implementation)
- Algorithm #5: an algorithm that handles maximal relationships and is designed explicitly for Monte Carlo simulation. Three versions of Algorithm #5 have been developed in this paper based on Algorithms #2, #3, and #4.

7. The algorithms

The following notations are used for the algorithms:

μ_i	–	is the distance label of node i .
S	–	the set of nodes whose distance label is optimal. Initially, S contains only the start node with label distance 0.
S^c	–	is the complement of S and represents the nodes whose distance label is not optimal.
s, t	–	are the start and finish points
A	–	is the set of the arcs
N	–	is the set of the nodes. ($S \cup S^c = N$)
d_{ij}	–	is the length of the arc (i,j) .

7.1. Algorithm #1

Algorithm #1 supposes networks without maximal relationships, one start and one finish, with relationships converted into their SS equivalent. (See Fig. 4d without edge from C_s to B_s , which causes loops.) The pseudo-code of the algorithm is presented below. This algorithm defines both the earliest and latest start policies. Finishes can be calculated after finishing the algorithm by simply adding the activity durations to the start dates. Every arc has to be examined exactly once in this network. The complexity of this algorithm is $O(m)$. [31].

7.2. Algorithm #2

Algorithm #2 can handle non-positive value loops; therefore, it can be used when maximal relationships are in the network (see Fig. 4c). This algorithm successfully updates the distance labels until they satisfy [2]. Satisfying [2], due to the algorithm's construction, is both a necessary and sufficient optimality condition.

algorithm #1 *longest path for acyclic networks (Kelley and Walker);*

```

begin
   $S := s; S^c := N \setminus s;$ 
   $\mu_i := -\infty \forall i \in S^c;$ 
   $\mu_i := 0;$  /* see [3]
  while  $|S^c| > 0$  do
    begin
      select a  $j \in S^c$  node for which every  $(i,j) \in A$  and  $i \in S$ 
      for each  $(i,j) \in A$ 
        if [2] not true then  $\mu_j := \mu_i + d_{ij};$ 
       $S^c := S^c - \{j\};$ 
       $S := S \cup \{j\};$ 
    end;
  end;

```

Algorithm #2;

```

begin
   $\mu_s := 0;$  /* see [3]
   $\mu_j := -\infty \forall j \in N - \{s\};$ 
   $changes = true;$ 
  while  $changes = true$ 
    begin
       $changes = false$ 
      do for all the arcs  $\in A$  /* Use any predefined order of arcs
        begin
          if [2] not true for the arc in question
            begin
               $\mu_j := \mu_i + d_{ij};$ 
               $changes = true;$ 
              take the next arc;
            end;
          end;
        end;
      end;
    end;
  end;

```

The worst-case complexity of this algorithm is $O(nm)$. This unpleasant scenario can arise in a one chain network with the worst possible sequence of relationships. This algorithm has many different modifications and improvements; in this paper, we investigate three modifications.

7.3. Algorithm #3 (FIFO implementation)

A vector containing nodes, called LIST, is introduced in this algorithm and maintained during the calculation. In the beginning, LIST contains only node s . In the next step, the node is taken out from the front of the LIST (node i), and the algorithm examines all (i, j) arcs. If arcs (i, j) violate [2], and j is not in the LIST, then j has to be put to the end of vector LIST. This method is called the first-in, first-out (FIFO) order. If LIST is empty, then the solution is optimal.

A C++ built-in container called VECTOR was used for FIFO implementation. This algorithm's worst-case complexity is $O(nm)$, the same as for algorithm #2. However, the authors hypothesize that algorithm #3 is faster than algorithm #2 for construction networks with maximal relationships.

7.4. Algorithm #4 (DEQUEUE implementation)

This algorithm is similar to algorithm #3. The difference lies in the way vector LIST is created. If a node j has to be included in the LIST (because [2] is not satisfied) and earlier it was in the list, the algorithm puts it to the front. Otherwise, the algorithm puts it to the end. This implementation was solved by DEQUEUE built-in C++ container. A dequeue is a data structure that permits us to store a list and quickly add or delete elements from the front and the rear of the list.

```

algorithm #3 (FIFO implementation);
begin
     $\mu_s := 0;$  /* see [3]
     $\mu_j := -\infty \quad \forall j \in \mathbf{N} - \{s\};$ 
    LIST := {s};
    while LIST  $\neq \emptyset$  do
        begin
            remove an element  $i$  from the front of the LIST;
            for each arc  $(i, j) \in A(i)$  do
                if [2] is not true then
                    begin
                         $\mu_j := \mu_i + d_{ij};$ 
                        if  $j \notin \text{LIST}$  then add node  $j$  to the end of the LIST;
                    end;
            end;
        end;
    end;

```

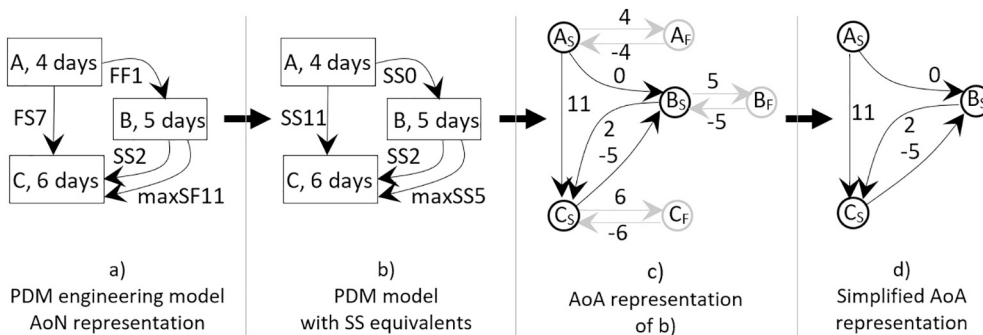


Fig. 5. Converting the engineering model into an equivalent mathematical model.

algorithm #4 (DEQUEUE implementation);
begin
 $\mu_s := 0;$ /* see [3]
 $\mu_j := -\infty \quad \forall j \in \mathbf{N} - \{s\};$
LIST:= {s};
while LIST $\neq \emptyset$ **do**
 begin
 remove an element i from the front of the LIST;
 for each arc $(i, j) \in A(i)$ **do**
 if [2] **is not true then**
 begin
 $\mu_j := \mu_i + d_{ij};$
 if $j \in \text{LIST}$
 then add node j to the front of the LIST
 else add node j to the end of the LIST;
 end;
 end;
 end;
end;

The worst-case complexity of the algorithm with dequeue implementation is $O(nm)$, the same as in algorithm #2 and #3. However, it is hypothesized that it examines fewer nodes in linear construction networks than the algorithm with FIFO implementation (Algorithm #3).

Algorithms #2, #3, and #4 can handle cycles in the network. Cycles are the results of transforming maximal relationships into minimal relationships. A minFSx and a maxFSy relationship between the same two activities create a loop when maxFSy is transformed into its minimal equivalent. These relationships contradict each other if $y < x$; in these cases, the loops' value will be positive. A network with positive value loops is unsolvable. Algorithms #2-#4 examine each node at most $(n-1)$

times, where n is the number of nodes. If a node is examined more than $(n-1)$ times, it indicates that at least one positive value loop is in the network, and the algorithms stop.

7.5. Algorithms #5 (Two-phase algorithms)

Algorithm #5/a, #5/b, and #5/c are designed explicitly for Monte Carlo simulation. Algorithm #5/a employs Algorithm #2 for the first instance, with the only difference that the calculation sequence of arcs is stored in an array called LIST_ARC. This calculation order is used for the rest of the instances. Algorithms #5b and #5c do the same with the

Table 2
Comparison of algorithms.

	Algorithm						
	#1	#2	#3	#4	#5a	#5b	#5c
Network structure	Acyclic	Non-positive loops are allowed	Non-positive loops are allowed	Non-positive loops are allowed	Non-positive loops are allowed	Non-positive loops are allowed	Non-positive loops are allowed
Worst case complexity	$O(m)$	$O(nm)$	$O(nm)$	$O(nm)$	$O(kmn)^a$	$O(kmn)^a$	$O(kmn)^a$
Origins	[22]	[4,12]	FIFO Bellmann 1958 Moore [37]	DEQUEUE [30,1]	Result of this research	Result of this research	Result of this research
Feature	Traditional CPM/PDM	PDM with maximal relationships	PDM with maximal relationships	PDM with maximal relationships	Monte Carlo sim. based on Alg. #2	Monte Carlo sim. based on Alg. #3	Monte Carlo sim. based on Alg. #4

^a k is the number of instances in Monte Carlo simulation.

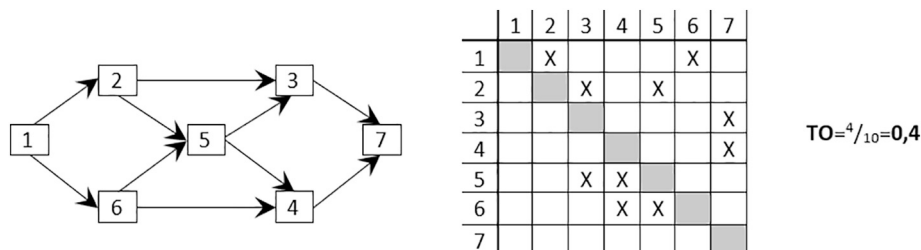


Fig. 6. Interpretation of topological orderliness.

difference that #5/b uses Algorithm #3, while #5c uses Algorithm #4 for the first instances.

For example, in Algorithm #5a, arcs not satisfying [2] are stored in a vector called LIST_ARCS after finishing the calculations for the first instance (1st phase). In the second phase, LIST_ARC is used to calculate the rest of the instances. The algorithm goes through all the elements of LIST_ARC and calculates E_j values based on the length of the arc (i, j) and E_i , regardless of whether [2] is satisfied or not.

Based on the above, developing Algorithm #5b and #5c is simple, and one can construct them easily.

```

algorithm #5a (two-phase algorithm based on Algorithm #2);
begin
  /* 1 st phase: First instance of the Monte Carlo analysis
   $\mu_s := 0$ ; /* see [3]
   $\mu_j := -\infty$  for each  $j \in \mathbf{N} - \{s\}$ ;
  LIST := { s }
  LIST_ARC := empty
  while LIST  $\neq$  0 do
    begin
      remove an element  $i$  from the front of the LIST;
      for each arc  $(i, j) \in A(i)$  do
        if [2] is not true then
          begin
             $\mu_j := \mu_i + d_{ij}$ ;
            add node  $j$  to the end of the LIST;
            add arc  $(ij)$  to the end of the LIST_ARC;
          end;
        end;
      end;
  /* 2nd phase: From the second instance of the Monte Carlo analysis
   $\mu_s := 0$ ; /* see [3]
   $\mu_j := -\infty$  for each  $j \in \mathbf{N} - \{s\}$ ;
  for each arc  $(i, j) \in \text{LIST\_ARC}$  do in ascending order
     $\mu_j := \mu_i + d_{ij}$ ;
end;
  
```

7.6. Comparison of the algorithms

Information regarding the algorithms is summarized in the table below. (Table 2).

With the introduction of the engineering/mathematical models and the five algorithms' pseudo-codes, research objective #1 is fulfilled.

8. The characteristics of linear construction networks

How a particular algorithm for resource planning performs on networks with different topologies raises network complexity issues. Complexity Indices (CIs) of network schedules have been the subject of many papers since the sixties [3]. Despite the still open debate 'whether the complexity is an inborn feature of a project network or an effect of a particular network analysis or computer implementation' [28], Complexity Indices are considered to be potentially useful tools. However, to the best of these authors' knowledge, no CI exists to aid selecting among algorithms for time analysis for different networks.

The following indicators are thought to be relevant from the research perspective:

I_1 - What is the ratio of the number of relationships (m) and the number of activities (n)? (m/n)

I_2 - What is the ratio of the number of maximal relationships ($maxm$)

and relationships (m)?

I_3 - What is the topological orderliness (TO) of activities in the database?

The complexity of Algorithms #2- #5 is $O(nm)$, which means that these algorithms run faster on sparse graphs. A graph is considered sparse if its edges' number is much less than the possible number of edges. The ratio of m to n (I_1) can capture sparsity. The term sparse network in this context defines a PDM schedule network where I_1 is less than five.

Maximal relationships significantly affect the algorithm's speed since Algorithm #1 is not applicable in this case; therefore, the index based on the ratio of $maxm$ to m is also defined.

Interpretation of topological orderliness is supported by Fig. 6. Let us assume that activity IDs follow the input sequence. Topological orderliness is defined for this paper as the ratio of the number of relationships below the adjacency matrix's diagonal to the total number of relationships (Fig. 6). The Xs in the matrix represent the ten relationships. There are four relationships below the diagonal, which means that the network's topological orderliness in Fig. 6 is 0.4. It is assumed that activities in a construction schedule network more or less follow their topological order. The closer this index to zero, the higher the topological orderliness is.

Twenty real-life schedule networks of highway and railway projects were examined to define the value of the indices described above. Table 3 shows the relevant data for the sample networks. Table 4 shows the averages and standard deviations for each indicator.

The following observations can be concluded from Table 3:

- Graph equivalents of highway and railway schedule networks are always sparse graphs. In some of the studied cases, the number of relationships seemed to be low compared to the number of activities, probably due to an incomplete project logic. This belief was also strengthened by the relatively large number of constraints and sometimes open ends. However, even with a higher number of relationships, all graphs would be considered sparse.

Table 3
Topological indicators of construction schedule networks.

ID.	n	n	maxm	I ₁	I ₂	I ₃	Remarks
RL001	258	260	0	1.01	0	0.04	Railway
RL002	51	57	0	1.12	0	0.12	Railway, Milestone network
RL003	839	1052	0	1.25	0	0.04	Highway
RL004	221	310	0	1.40	0	0.03	Highway
RL005	734	1166	0	1.59	0	0.03	Junction.
RL006	1295	2731	3	2.11	0	0.05	Highway new
RL007	339	520	0	1.53	0	0.04	Highway
RL008	503	507	0	1.01	0	0.04	Railway
RL009	409	572	0	1.40	0	0.04	Highway
RL010	141	147	0	1.04	0	0.00	Railway
RL011	494	532	0	1.08	0	0.05	Railway
RL012	268	439	0	1.64	0	0.04	Highway
RL013	210	305	0	1.45	0	0.06	Highway
RL014	347	591	0	1.70	0	0.03	Highway
RL015	374	523	0	1.40	0	0.03	Highway
RL016	564	766	0	1.36	0	0.02	Highway
RL017	1028	1420	2	1.38	0	0.03	Highway
RL018	492	780	0	1.59	0	0.02	Highway
RL019	934	1291	0	1.38	0	0.01	Highway
RL020	1310	2057	4	1.57	0	0.04	Highway

Table 4
Averages and standard deviations of the indexes.

	Average	Standard deviation
I ₁ (m/n)	~1.4	~0.26
I ₂ (maxm/m)	~0.01	~0.00
I ₃ (TO)	~0.04	~0.01

Table 5
PNGEN: Some of the essential controls and their domains.

Name of the control in PNGEN	Domain	Remark
Number of nodes/activities (n)	100–100,000	≥n
Number of minimal relationships	100–1,000,000	
Number of maximal relationships	0–100,000	
Lower and upper limits for activity durations	0–1,000	
Lower and upper limits for relationship durations (lags)	0–1,000	
Allowed relationships types	FS, SS, SF, FF	
Topological orderliness (TO) (see Fig. 5)	0–1	
Number of start nodes (sn)	1–1,000	sn + fn ≤ n
Number of finish nodes (fn)	1–1,000	sn + fn ≤ n

- The use of maximal relationships is sporadic. The authors believe that maximal relationships are essential in construction project schedules, and no project logic can be complete without maximal relationships. Unfortunately, prevalent applications do not support maximal relationships, and probably that is why their application of schedule networks is so rare.
- Activities in the database follow a chronographic/logical order; that is, preceding activities of any activity are usually in the database when the said activity is entered.
- Small standard deviations indicate that the above indexes can be used as indicators of linear-construction-like networks.

The Project Network Generator described in the next section uses these indexes to generate the artificial sample projects of any size of activities.

9. The project network generator

A network generator application (PNGEN) has been developed to generate schedule networks of any kind, including linear scheduling like construction networks. PNGEN can generate the mathematical equivalent (Fig. 5) of the engineering network by entering its characteristics. A

set of numerical options and switches have been built into the software to provide a variety of generated networks. This is summarized in Table 5.

After constructing the network, built-in routines attempt to calculate the early and late times. In case of success (negative loops are identified), elaborated tabular display provides deep insight into the generated logical structure. The paths of the network can be walked through both forward and backward. A verbal summary report is also displayed with a short statistical overview of different features and results after each run. Various data formats (different data structures in pure “txt” text format) are available for communicating other applications. [36]. The application and the respective user manual have been made publicly available for the research community through the http://www.ekt.bme.hu/Projlib_en.shtml link.

With the development of the PNGEN, research objective #2 has been fulfilled.

10. The project network library (PNLIB)

For this research, 50 artificial construction-like networks have been generated with PNGEN and placed into the Project Network Library database (PNLIB). The name of a network, as it is shown in the first column of Table 6, indicates if the network is artificial (AN) or real life (RL), followed by the number of activities in the network in five characters with the number of the network shown using four characters. Thus, the maximal number of a network's activities in PNLIB can vary between 1 and 99,999, and 9999 networks can be stored for a given activity number. The second column of Table 6 shows the test networks' activity numbers. Networks of 1000, 2000, 5000, 10,000, 20,000 activities were generated, ten in each group. The third column defines the ratio of relationships to activities (I₁). Each group contains two networks with a 1.4 relationship/activity ratio; there are six networks with I₁ = 1.6 and two networks with I₁ = 1.8 values. Instead of displaying I₂, the number of maximal relationships is shown in column #4. The fifth column of Table 6 shows the index related to topological orderliness. The authors plan to make this library publicly available so that other researchers can use the library files for their research.

By generating the 50 artificial networks and placing them into PNLIB, research objective #3 is fulfilled.

11. Numerical results

Table 6 shows the algorithms' relative speed on the fifty artificially created linear-construction-like schedule networks compared to the topologically fully ordered AN01000_0001 analyzed by Algorithm #1. Networks in .txt format are available at http://www.ekt.bme.hu/Projlib_en.shtml. The first column of the table shows the network's name; the second contains the number of activities; the third gives the ratio of the relationships to the activities; the fourth shows the number of maximal relationships. The fifth (I₃) is for the topological orderliness of the network. According to our experiences, perfect topological orderliness in schedule networks is rare. However, creating a topologically fully ordered one for the basis of comparison is useful. The network used for this purpose is called AN01000_0001, and it has 1000 activities and 1400 minimal traditional precedence relationships. There are no maximal relationships in the network, and it is topologically fully ordered. The running time of network AN01000_0001, based on the average of 1000 test runs, was $2.14 \cdot 10^{-5}$ s on the test computer. The running speed for the other cases is compared to this data. For instance, the relative speed 8973 of Algorithm #2 on network AN20000_0010 shows that, based on the average of one-thousand test runs, Algorithm #2 has spent $8973 \cdot 2.14 \cdot 10^{-5}$ that is 0.19 s to analyze this network once.

Monte Carlo simulation based on one-thousand instances on AN20000_010 using Algorithm 5/c takes only $138 \cdot 1000 \cdot 2.14 \cdot 10^{-5}$, that is, only 2.39 s. Introducing relative speeds for comparing the

Table 6

Test results.

ID	n	I ₁	maxm	I ₄	#1	#2	#3	#4	#5a	#5b	#5c
AN01000_0001	1000	1.4	0	0	1	1.5	6	1.1	1.1	2.5	1.4
AN01000_0002	1000	1.4	3	0.02		3	14	29	1.6	4.3	1.7
AN01000_0003	1000	1.6	3	0.03		6	15	53	1.7	4.6	1.6
AN01000_0004	1000	1.6	3	0.03		4	22	50	1.6	5.4	2.1
AN01000_0005	1000	1.6	3	0.04		6	19	53	1.9	5.9	1.7
AN01000_0006	1000	1.6	5	0.04		7	13	54	2.1	4.9	2.0
AN01000_0007	1000	1.6	5	0.04		5	5	9	1.7	2.4	2.5
AN01000_0008	1000	1.6	5	0.05		5	15	56	1.8	5.0	1.8
AN01000_0009	1000	1.8	5	0.06		8	27	154	2.3	6.6	2.6
AN01000_0010	1000	1.8	5	0.07		7	21	96	2.3	6.5	2.2
AN02000_0001	2000	1.4	0	0	2.4	3	42	154	2.4	17	7
AN02000_0002	2000	1.4	3	0.02		10	28	204	4	14	4
AN02000_0003	2000	1.6	3	0.03		19	71	386	6	39	3.4
AN02000_0004	2000	1.6	3	0.03		9	33	212	3	15	5
AN02000_0005	2000	1.6	3	0.04		18	60	374	5	21	7
AN02000_0006	2000	1.6	5	0.04		38	59	296	8	32	3.5
AN02000_0007	2000	1.6	5	0.04		30	59	395	6	31	3.6
AN02000_0008	2000	1.6	5	0.05		36	62	357	8	30	3.4
AN02000_0009	2000	1.8	5	0.06		39	65	518	9	33	3.7
AN02000_0010	2000	1.8	5	0.07		58	61	297	11	30	3.5
AN05000_0001	5000	1.4	0	0	7	8	245	1671	7	138	8
AN05000_0002	5000	1.4	3	0.02		84	270	2034	25	138	15
AN05000_0003	5000	1.6	3	0.03		146	501	2783	35	224	11
AN05000_0004	5000	1.6	3	0.03		205	530	3049	46	252	13
AN05000_0005	5000	1.6	3	0.04		193	401	4288	43	201	14
AN05000_0006	5000	1.6	5	0.04		231	397	2883	53	203	12
AN05000_0007	5000	1.6	5	0.04		214	529	2416	47	257	12
AN05000_0008	5000	1.6	5	0.05		276	694	4137	61	328	19
AN05000_0009	5000	1.8	5	0.06		401	863	3643	87	395	18
AN05000_0010	5000	1.8	5	0.07		462	761	4308	87	346	18
AN10000_0001	10,000	1.4	0	0	16	16	1073	6215	13	568	33
AN10000_0002	10,000	1.4	3	0.02		327	924	8604	87	520	24
AN10000_0003	10,000	1.6	3	0.03		656	2536	17,241	146	1362	32
AN10000_0004	10,000	1.6	3	0.03		766	2697	18,434	170	1535	41
AN10000_0005	10,000	1.6	3	0.04		942	2872	10,190	220	1664	43
AN10000_0006	10,000	1.6	5	0.04		877	2643	15,809	201	1539	31
AN10000_0007	10,000	1.6	5	0.04		967	2884	16,200	220	1662	33
AN10000_0008	10,000	1.6	5	0.05		1237	2619	13,112	277	1475	34
AN10000_0009	10,000	1.8	5	0.06		1459	3575	24,082	307	1990	43
AN10000_0010	10,000	1.8	5	0.07		2165	3771	19,262	444	2111	44
AN20000_0001	20,000	1.4	0	0	38	33	7910	28,510	27	4577	94
AN20000_0002	20,000	1.4	3	0.02		1369	6123	52,964	335	3960	104
AN20000_0003	20,000	1.6	3	0.03		3429	12,858	57,114	741	8178	102
AN20000_0004	20,000	1.6	3	0.03		3388	15,384	86,729	750	9708	110
AN20000_0005	20,000	1.6	3	0.04		4392	13,137	76,451	938	8368	124
AN20000_0006	20,000	1.6	5	0.04		4025	12,084	82,829	885	7757	102
AN20000_0007	20,000	1.6	5	0.04		4340	12,463	79,895	944	8153	102
AN20000_0008	20,000	1.6	5	0.05		5723	13,216	64,136	1244	8605	114
AN20000_0009	20,000	1.8	5	0.06		7525	16,882	85,997	1539	10,291	133
AN20000_0010	20,000	1.8	5	0.07		8973	18,505	81,141	1783	11,148	138

algorithms' speed creates the possibility to compare algorithms developed by others and tested on other computers.

Table 7 shows the average relative speeds compared to the speed of Algorithm #1. The averages shown in the cells are the averages of the values of the respective columns of Table 6. The values in these cells are useful to compare the speed of the algorithms to each other. For instance, Algorithm #5c outperforms Algorithm #2 around 15 times (1102/71).

With this, research objective #4 is considered to be fulfilled.

We kindly ask the reviewer to accept or modifications.

Table 7

Averages of the relative speed of the algorithms compared to the speed of Alg #1 on AN_01000-0001.

	Alg. #2	Alg. #3	Alg. #4	Alg. #5a	Alg. #5b	Alg. #5c
Averages of the relative speeds	1102	3300	17,597	236	2019	71

12. Discussion

The most important findings of the study in descending order are the following:

- Unsurprisingly, Algorithm #1 is the fastest since every relationship must be taken into account exactly once. Unfortunately, its use is limited to schedule networks containing only minimal relationships.
- In the case of maximal relationships, Algorithm #2 is the fastest for single time analysis, despite that Algorithm #3 and Algorithm #4 were hypothesized to be faster based on algorithms using similar approaches to the minimal path problem ([1], p 154)). Algorithm #2 running time takes one-third of Algorithm #3 and one-sixteenth of Algorithm #4.
- The explanation for this salient finding is that in project networks, relationships and activities are entered into the database more or less in the time analysis order; that is, schedule networks are usually topologically ordered. In this case, Algorithm #2 requires a relatively small number of iterations.

- The two-phase algorithms perform much better for Monte Carlo simulation than the single time analysis algorithms. The average running speed is four times faster than those based on the repetition of single time analysis algorithms.
- Algorithm 5/c is the fastest for Monte Carlo simulation among the studied two-phase algorithms. On average, it performs sixteen times better than Algorithm #2, almost fifty times better than Algorithm #3, and around 250 times better than Algorithm #5. It outperforms Algorithm #5/a more than three times and Algorithm #5/b around twenty-eight times. This result is surprising since the algorithm based on DEQUEUE implementation (Algorithm #4) proved to be the slowest for single time analysis.
- Calculating 1000 instances of Monte Carlo simulation on a network with 20,000 activities and 36,000 precedence relationships would take around 3 s using Algorithm #5/c (CPU I5-4570, 16GB RAM, Visual Studio C++, Windows 10). This means that #5/c solves large projects one-two orders of magnitude faster than the algorithms in some well-known commercial applications.

Among the limitations of this study, it must be mentioned that only twenty real-life networks were used to capture the typical features of linear construction schedule networks. While this set of samples is not representative, the hypotheses that linear schedule networks are sparse, have only a few maximal relationships, and their input order follows the networks' topological order seems acceptable, based on experience. Also, it seems to be true for all types of construction projects. Nevertheless, further research is necessary to verify these hypotheses. Also, possible other characteristics can exist to describe linear or other construction projects, and some of these characteristics may affect the algorithms' speed.

Other limitations regarding the model were discussed in the section 'Research goals and limitations'. The effects of these limitations on the algorithm's speed were not studied. This can be the direction of some future research. Also, during this study, it became clear that further improvements can be made regarding the efficiency of the algorithm, especially in the cases of Algorithms #2–4, by considering relationships in different orders resulting in fewer iterations. More efficient data structure and other IT connected solutions (for example, using all the cores of the processor) can also increase the algorithms' speed. With this, research objective #5 is fulfilled.

13. Conclusions and future research

The speed of different time analysis algorithms was studied in this paper to find the most suitable for analyzing construction-like schedule networks containing maximal relationships. Three algorithms (Algorithms #2–4) have, and their modifications for MC simulation (Algorithms #5/a–c) have been implemented for computers. Twenty real-life highway and railway construction projects were studied to capture the typical features of networks of linear construction. Research has shown that linear construction schedules are sparse, topologically well-ordered, and the number of loops (caused by maximal relationships) is small. An application called PNGEN and a database called PNLIB were developed to generate and store linear-construction-like test networks. The algorithms were implemented in C++ (Visual Studio). The algorithms' speed was tested on 50 artificially created networks stored in PNLIB.

Results show that Algorithm #2 performs much better than Algorithm #3 and #4, although these latter were expected to be faster. For Monte Carlo analysis, Algorithm #5/c (based on DEQUEUE implementation) performs better than Algorithm #5/a and Algorithm #5/b. The most important practical consequence of this study is that Monte Carlo simulation for large construction-like networks containing maximal relationships can be performed within a couple of seconds.

A possible future research direction is to improve these algorithms' speed further. Specially designed algorithms can be created for bi-

directional relationships since producing a new sequence for teams only partly changes the graph's topology.

PNLIB and PNGEN have been made publicly available to enable researchers to join and compare their results to those of this paper.

Data availability statement

Real-life networks (RL001–RL020) used for capturing the characteristics of construction networks are available from the authors upon request. PNGEN, the tool that generates artificial networks is available at http://www.ekt.bme.hu/Projlib_en.shtml. The benchmark database is available at http://www.ekt.bme.hu/Projlib_en.shtml. All other data generated or analyzed during the study are included in the published paper.

Declaration of Competing Interest

We have no conflicts of interest to disclose.

Acknowledgment

The research reported in this paper has been supported by the National Research, Development, and Innovation Fund (TUDFO/51757/2019-ITM, Thematic Excellence Program).

References

- [1] R.K. Ahuja, T.L. Magnanti, J.B. Orlin, *Network flows, Theory, Algorithms and Applications*, Prentice Hall, New Jersey, 1993 [ISBN-13: 978-0136175490].
- [2] D. Ajwani, et al., An empirical comparison of k-shortest simple path algorithms on multicores, in: *Proceedings of the 47th International Conference on Parallel Processing*, 2018, pp. 1–12, <https://doi.org/10.1145/3225058.3225075>.
- [3] D. Baccarini, The concept of project complexity — A review, *Int. J. Proj. Manag.* 14 (4) (1996) 201–204, [https://doi.org/10.1016/0263-7863\(95\)00093-3](https://doi.org/10.1016/0263-7863(95)00093-3).
- [4] R. Bellman, On a routing problem, *Q. Appl. Math.* 16 (1) (1958) 87–90, <https://doi.org/10.1090/qam/102435>.
- [5] E.W. Dijkstra, A note on two problems in connexion with graphs, *Numer. Math.* 1 (1) (1959) 269–271, <https://doi.org/10.1007/bf01386390>.
- [6] S.E. Elmaghraby, J. Kamburowski, The Analysis of Activity Networks Under Generalized Precedence Relations (GPRs), *Manag. Sci.* 38 (9) (1992) 1245–1263, <https://doi.org/10.1287/mnsc.38.9.1245>.
- [7] J.W. Fondahl, *A Non-Computer Approach to the Critical Path Method for the Construction Industry*, Technical Report #9, 2nd edition, Department of Civil Engineering, Stanford University, Stanford, CA, 1962, p. 155, <http://hdl.handle.net/2027/mdp.39015000453970>. Accessed at: 03/05/2020.
- [8] J.W. Fondahl, The history of modern project management: precedence diagramming method: Origins and early developments, *Proj. Manag. J.* 18 (2) (1987) 33–36 [ISSN 1938-9507] Accessed at: <https://www.pmi.org/learning/library/precedence-diagramming-methods-origins-development-5222> on May 3, 2020.
- [9] A. Francis, E.T. Miresco, Decision support for project management using a chronographic approach, *J. Decis. Syst.* 11 (3–4) (2002) 383–404, <https://doi.org/10.3166/jds.11.383-404>.
- [10] L.R. Ford Jr., *Network Flow Theory*. Paper P-923, RAND Corporation, Santa Monica, California, 1956. Accessed at: <https://apps.dtic.mil/dtic/tr/fulltext/u2/422842.pdf> on May 3, 2020.
- [11] J.F. Gilsinn, A Performance Comparison of Labeling Algorithms for Calculating Shortest Path Trees, 1973, p. 96, <https://doi.org/10.6028/nbs.tn.772>.
- [12] M. Hajdu, *Network Scheduling Techniques for Construction Project Management*. Nonconvex Optimization and Its Applications, Kluwer Academic Publishers, 1997, p. 352, <https://doi.org/10.1007/978-1-4757-5951-8>.
- [13] M. Hajdu, One relation to rule them all: The point-to-point precedence relation that substitutes the existing ones, in: T.M. Froese, L. Newton, F. Sadeghpour, D. J. Vanier (Eds.), (2015). *Proceedings of ICSC15: The Canadian Society for Civil Engineering 5th International/11th Construction Specialty Conference*, University of British Columbia, Vancouver, Canada, 2015, <https://doi.org/10.14288/1.0076408>. June 7–10, pp. 340 1–11.
- [14] M. Hajdu, History and Some Latest Developments of Precedence Diagramming Method, *Organiz. Technol. Manag. Construct. Int. J.* 7 (2) (2015) 1302–1314, <https://doi.org/10.5592/otmcj.2015.2.5>.
- [15] M. Hajdu, Continuous precedence relations for better modelling overlapping activities, *Procedia Eng.* 123 (2015) 216–223, <https://doi.org/10.1016/j.proeng.2015.10.080>.
- [16] M. Hajdu, PDM time analysis with continuous and point-to-point relations: calculations using an artificial example, *Procedia Eng.* 164 (2016) 57–67, <https://doi.org/10.1016/j.proeng.2016.11.592>.

- [17] M. Hajdu, Survey of precedence relationships: classification and algorithms, *Autom. Constr.* 95 (2018) 245–259, <https://doi.org/10.1016/j.autcon.2018.08.012>.
- [18] T. Hegazy, E. Kamarah, Efficient repetitive scheduling for high-rise construction, *J. Constr. Eng. Manag.* 134 (4) (2008) 253–264, [https://doi.org/10.1061/\(asce\)0733-9364\(2008\)134:4\(253\)](https://doi.org/10.1061/(asce)0733-9364(2008)134:4(253)).
- [19] K. Hiary, K. El-Rayes, Optimal planning and scheduling for repetitive construction projects, *J. Manag. Eng.* 22 (1) (2006) 11–19, [https://doi.org/10.1061/\(ASCE\)0742-597X\(2006\)22:1\(11\)](https://doi.org/10.1061/(ASCE)0742-597X(2006)22:1(11)).
- [20] IBM, Users' Manual for IBM 1440 Project Control System (PCS), Available at: http://ekt.bme.hu/Cikkek/12_27_1966_IBM.pdf, 1964. Latest access: March 1, 2021.
- [21] P.G. Ioannou, I.-T. Yang, Repetitive scheduling method: requirements, modeling, and implementation, *J. Constr. Eng. Manag.* 142 (5) (2016) 04016002, [https://doi.org/10.1061/\(asce\)co.1943-7862.0001107](https://doi.org/10.1061/(asce)co.1943-7862.0001107).
- [22] J.E. Kelley, M.R. Walker, Critical-path planning and scheduling, in: *Papers Presented at the December 1-3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference on - IRE-AIEE-ACM'59 (Eastern)*, 1959, <https://doi.org/10.1145/1460299.1460318>.
- [23] J.E. Kelley, Critical-path planning and scheduling: mathematical basis, *Oper. Res.* 9 (3) (1961) 296–320, <https://doi.org/10.1287/opre.9.3.296>.
- [24] J.A.G.M. Kerbosch, H.J. Schell, Network Planning by the Extended Metra Potential Method (EMPM) TH Eindhoven. ORS, Vakgr. operationele research: rapport, vol. KS-1.1, Technische Hogeschool Eindhoven. <https://pure.tue.nl/ws/portalfiles/portal/4368315/104181.pdf>, 1975.
- [25] S.-G. Kim, CPM schedule summarizing function of the Beeline Diagramming Method, *J. Asian Architect. Build. Eng.* 11 (2) (2012) 367–374, <https://doi.org/10.3130/jaabe.11.367>.
- [26] R. Kolisch, A. Sprecher, PSPLIB – A project scheduling problem library, *Eur. J. Oper. Res.* 96 (1) (1997) 205–216, [https://doi.org/10.1016/S0377-2217\(96\)00170-1](https://doi.org/10.1016/S0377-2217(96)00170-1).
- [27] K.G. Mattila, D.M. Abraham, Resource leveling of linear schedules using integer linear programming, *J. Constr. Eng. Manag.* 124 (3) (1998) 232–244, [https://doi.org/10.1061/\(asce\)0733-9364\(1998\)124:3\(232\)](https://doi.org/10.1061/(asce)0733-9364(1998)124:3(232)).
- [28] K.M. Nassar, M.Y. Hegab, Developing a complexity measure for project schedules, *J. Constr. Eng. Manag.* 132 (6) (2006) 554–561, [https://doi.org/10.1061/\(asce\)0733-9364\(2006\)132:6\(554\)](https://doi.org/10.1061/(asce)0733-9364(2006)132:6(554)).
- [29] G.B. Roy, Graphes et ordonnancement (Graphs and scheduling), in: *Revue Française de Recherche Opérationnelle n° 25, 4e trimestre, 1962*, pp. 323–333. http://ekt.bme.hu/Cikkek/606-2021_Roy_1962.pdf. Latest access: March 9, 2021.
- [30] U. Pape, Implementation and efficiency of Moore-algorithms for the shortest route problem, *Math. Program.* 7 (1) (1974) 212–222, <https://doi.org/10.1007/bf01585517>.
- [31] M. Thorup, Undirected single-source shortest paths with positive integer weights in linear time, *J. ACM (JACM)* 46 (3) (1999) 362–394, <https://doi.org/10.1145/316542.316548>.
- [32] D. Xie, et al., Practical private shortest path computation based on Oblivious Storage, in: *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, 2016, <https://doi.org/10.1109/icde.2016.7498254>.
- [33] X.Z. Wang, The comparison of three algorithms in shortest path issue, *J. Phys. Conf. Ser.* 1087 (2018) 022011, <https://doi.org/10.1088/1742-6596/1087/2/022011>.
- [34] M. Vanhoucke, J. Coelho, An approach using SAT solvers for the RCPSP with logical constraints, *Eur. J. Oper. Res.* 249 (2) (2016) 577–591, <https://doi.org/10.1016/j.ejor.2015.08.044>.
- [35] R.M. van Slyke, Letter to the editor—Monte Carlo methods and the PERT problem, *Oper. Res.* 11 (5) (1963) 839–860, <https://doi.org/10.1287/opre.11.5.839>.
- [36] Z.A. Vattai, Floyd-Warshall in scheduling open networks, *Procedia Eng.* 164 (2016) 106–114, <https://doi.org/10.1016/j.proeng.2016.11.598>.
- [37] E.F. Moore, The shortest path through a maze.. *Proc. International Symposium. Switching Theory 1957, Part II*, Harvard University Press, Cambridge, Massachusetts, 1959, pp. 285–292. MR 114710.